HashMap and HashSet Study Guide

1. Introduction to HashMap

What is HashMap?

HashMap is a data structure in Java that stores data in key-value pairs. It belongs to the Java Collections Framework and implements the Map interface. HashMap allows you to store and retrieve data efficiently using keys.

Key Characteristics:

- **Key-Value Pairs**: Each element consists of a key and its corresponding value
- **No Duplicate Keys**: Keys must be unique, but values can be duplicated
- **Null Values**: Allows one null key and multiple null values
- **Unordered**: Does not maintain insertion order
- **Fast Access**: Average O(1) time complexity for basic operations

Basic Syntax:

```java
HashMap<KeyType, ValueType> mapName = new HashMap<>();
```

Example:

```java
import java.util.HashMap;

HashMap<String, Integer> studentGrades = new HashMap<>();
studentGrades.put("Alice", 85);
studentGrades.put("Bob", 92);
studentGrades.put("Charlie", 78);

System.out.println(studentGrades.get("Alice")); // Output: 85
```

Common Methods:

- `put(key, value)` - Adds a key-value pair
- `get(key)` - Retrieves value by key

- `remove(key)` - Removes a key-value pair
- `containsKey(key)` - Checks if key exists
- `containsValue(value)` - Checks if value exists
- `size()` - Returns number of key-value pairs
- `isEmpty()` - Checks if HashMap is empty
- `clear()` - Removes all elements

2. Usage and Application

Basic Operations Example:

```java
import java.util.HashMap;

public class HashMapExample {
    public static void main(String[] args) {
        // Creating HashMap
        HashMap<String, String> countries = new HashMap<>();

        // Adding elements
        countries.put("USA", "Washington D.C.");
        countries.put("France", "Paris");
        countries.put("Japan", "Tokyo");
        countries.put("India", "New Delhi");

        // Accessing elements
        System.out.println("Capital of France: " + countries.get("France"));

        // Checking if key exists
        if (countries.containsKey("USA")) {
            System.out.println("USA is in the map");
        }

        // Removing an element
        countries.remove("Japan");

        // Iterating through HashMap
        for (String country : countries.keySet()) {
            System.out.println(country + " -> " + countries.get(country));
        }

        // Size of HashMap
        System.out.println("Size: " + countries.size());
    }
}
```

Real-World Applications:

1. Counting Frequency

```java
import java.util.HashMap;

public class FrequencyCounter {
    public static void main(String[] args) {
        String text = "hello world hello";
        String[] words = text.split(" ");

        HashMap<String, Integer> wordCount = new HashMap<>();

        for (String word : words) {
            wordCount.put(word, wordCount.getOrDefault(word, 0) + 1);
        }

        System.out.println(wordCount); // Output: {world=1, hello=2}
    }
}
```

2. Student Database

```java
import java.util.HashMap;

public class StudentDatabase {
    public static void main(String[] args) {
        HashMap<Integer, String> students = new HashMap<>();

        // Adding students
        students.put(101, "Alice Johnson");
        students.put(102, "Bob Smith");
        students.put(103, "Charlie Brown");

        // Finding student by ID
        int searchId = 102;
        if (students.containsKey(searchId)) {
            System.out.println("Student found: " + students.get(searchId));
        } else {
            System.out.println("Student not found");
        }
    }
}
```

3. Caching System

```java
import java.util.HashMap;

public class SimpleCache {
    private HashMap<String, String> cache = new HashMap<>();

    public void put(String key, String value) {
        cache.put(key, value);
        System.out.println("Cached: " + key);
    }

    public String get(String key) {
        if (cache.containsKey(key)) {
            System.out.println("Cache hit for: " + key);
            return cache.get(key);
        } else {
            System.out.println("Cache miss for: " + key);
            return null;
        }
    }
}
```

Introduction to HashSet

HashSet is a collection that stores unique elements. It's built on top of HashMap internally.

Key Characteristics:

- **No Duplicates**: Automatically handles duplicate removal
- **Unordered**: Does not maintain insertion order
- **Null Values**: Allows one null element
- **Fast Operations**: Average O(1) time complexity

Basic HashSet Example:

```java
import java.util.HashSet;

public class HashSetExample {
    public static void main(String[] args) {
        HashSet<String> fruits = new HashSet<>();

        // Adding elements
        fruits.add("Apple");
        fruits.add("Banana");
        fruits.add("Orange");
        fruits.add("Apple"); // Duplicate - will be ignored

        System.out.println(fruits); // Output: [Apple, Banana, Orange]
        System.out.println("Size: " + fruits.size()); // Output: 3

        // Checking if element exists
        if (fruits.contains("Apple")) {
            System.out.println("Apple is in the set");
        }

        // Removing element
        fruits.remove("Banana");
        System.out.println("After removal: " + fruits);
    }
}
```

3. Practice Questions on HashMap

Question 1: Basic HashMap Operations

```java
import java.util.HashMap;

public class Practice1 {
    public static void main(String[] args) {
        HashMap<String, Integer> inventory = new HashMap<>();
        inventory.put("Apples", 50);
        inventory.put("Bananas", 30);
        inventory.put("Oranges", 25);

        // What will be printed?
        System.out.println(inventory.get("Bananas"));
        System.out.println(inventory.size());
        System.out.println(inventory.containsKey("Grapes"));
    }
}
```

Question 2: Updating Values

```java
import java.util.HashMap;

public class Practice2 {
    public static void main(String[] args) {
        HashMap<String, Integer> scores = new HashMap<>();
        scores.put("Player1", 100);
        scores.put("Player2", 150);
        scores.put("Player1", 200); // Updating existing key

        // What will be the output?
        System.out.println(scores.get("Player1"));
        System.out.println(scores.size());
    }
}
```

Question 3: Null Handling

```java
import java.util.HashMap;

public class Practice3 {
    public static void main(String[] args) {
        HashMap<String, String> map = new HashMap<>();
        map.put("key1", "value1");
        map.put("key2", null);
        map.put(null, "nullKey");

        // What will be printed?
        System.out.println(map.get("key2"));
        System.out.println(map.get(null));
        System.out.println(map.containsKey(null));
    }
}
```

Quiz Questions

1. What is the default initial capacity of a HashMap in Java?

a) 8

b) 16

c) 32

d) 64

Answer: b) 16

2. Which method is used to check if a HashMap contains a specific key?

a) hasKey()

b) containsKey()

c) checkKey()

d) findKey()

Answer: b) containsKey()

3. What happens when you try to put a duplicate key in a HashMap?

a) It throws an exception

b) It creates a new entry

c) It updates the existing value
d) It ignores the operation

Answer: c) It updates the existing value

4. How many null keys can a HashMap store?

a) 0
b) 1
c) Multiple
d) Unlimited

Answer: b) 1

5. What is the average time complexity for basic operations (get, put, remove) in HashMap?

a) O(1)
b) O(log n)
c) O(n)
d) $O(n^2)$

Answer: a) O(1)

Practice Coding Questions

1. Word Frequency Counter

Write a Java program that takes a sentence as input and counts the frequency of each word using HashMap. The program should be case-insensitive.

2. Student Grade Manager

Create a program that uses HashMap to store student names as keys and their grades as values. Implement methods to:

- Add a new student with grade
- Update an existing student's grade
- Find the highest grade
- Display all students with grades above 80

3. Character Frequency in String

Write a program that counts the frequency of each character in a given string using HashMap. Ignore spaces and make it case-insensitive.

4. Remove Duplicates using HashSet

Create a program that takes an array of integers and removes duplicates using HashSet. Return the unique elements as a new array.

5. Two Sum Problem

Given an array of integers and a target sum, write a program using HashMap to find two numbers in the array that add up to the target. Return their indices.